

Day 17 — Advanced GitHub Actions

Day 17 — Advanced GitHub Actions Lab

Objective

Practice job dependencies, conditional execution, matrix builds, secrets, artifacts, and caching in real workflows.

Mental Model

Workflow -> Jobs -> Runners -> Steps

needs -> controls job dependency
if -> controls conditional execution
matrix -> runs multiple configurations
secrets -> sensitive values
artifacts-> build outputs
cache -> performance optimization

Tasks

1. Job dependencies

Use `../../examples/job-dependencies.yml`. Confirm in the Actions UI that test only starts after build finishes, and deploy only after test.

2. Conditional execution

Use `../../examples/conditions.yml`. Push to develop and confirm only deploy-staging runs. Push to main and confirm only deploy-production runs.

3. Matrix strategy

Use `../../examples/matrix-strategy.yml`. Count how many jobs the matrix produces (os x node-version) and explain `fail-fast: false`.

4. Secrets

In your repo: **Settings -> Secrets and variables -> Actions**, add a secret named `API_TOKEN` with a dummy value. Run

`../../examples/secrets.yml` and confirm the value never appears in logs.

5. Artifacts

Run `../../examples/artifacts.yml`. Download the `dist-files` artifact from the Actions run summary and confirm it contains `app.txt`.

6. Caching

Run `../../examples/caching.yml` twice. Compare the run time and the "Cache restored from key" log line between the first (cache miss) and second (cache hit) run.

7. Permissions

Add a `permissions:` block scoped to only what each workflow needs (`contents:` `read`, `packages:` `write`, etc.) instead of relying on defaults.

Success Criteria

- ☐ Can explain needs and draw the resulting job graph
- ☐ Can explain when an `if:` condition evaluates to true/false
- ☐ Can explain what a matrix expands into
- ☐ Secrets never appear in plaintext in logs
- ☐ Artifacts successfully passed between jobs
- ☐ Cache hit confirmed on a second run

Day 17 Interview Questions — Advanced GitHub Actions

Q1. What does `needs` do?

Declares that a job depends on one or more other jobs completing successfully first, turning independent parallel jobs into a pipeline.

Q2. How do you conditionally run a job or step?

With an `if:` expression, e.g. `if: github.ref == 'refs/heads/main'` or `if: failure()` to run only when a prior step/job failed.

Q3. What is a matrix strategy used for?

Running the same job across multiple configurations (OS versions, language versions, etc.) in parallel — `strategy: matrix:` expands into one job per combination.

Q4. What does `fail-fast: false` do in a matrix?

Normally, one matrix job failing cancels all the others. `fail-fast: false` lets every matrix combination run to completion independently.

Q5. What are GitHub Secrets?

Encrypted values stored at the repo, environment, or organization level and referenced via `${{ secrets.NAME }}`; GitHub automatically masks their values in logs.

Q6. Secrets vs environment variables — what's the difference?

Environment variables (`env:`) are plain values visible in the workflow file/logs. Secrets are encrypted at rest and masked in logs — always use secrets for credentials, tokens, and keys.

Q7. What is a GitHub Environment?

A named deployment target (e.g. `production`) that can have protection rules: required reviewers, wait timers, and scoped secrets.

Q8. What are artifacts and when do you need them?

Files produced by one job that need to be used by another job (or kept after the run) — since each job runs on an isolated runner, artifacts are the explicit way to pass files between them.

Q9. What does caching optimize, and how is a cache key typically built?

Speeds up repeated dependency installs by reusing previously downloaded packages. Keys are usually built from a hash of the lockfile, e.g.

`${{ hashFiles('**/package-lock.json') }}`, so the cache invalidates

automatically when dependencies change.

Q10. Why scope permissions: explicitly instead of using the default?

Least privilege — the default `GITHUB_TOKEN` can be broader than a workflow needs; explicitly setting `permissions: contents: read (etc.)` limits the blast radius if a workflow or an Action it uses is compromised.

Q11. What's a supply-chain risk with third-party Actions?

Pinning to a mutable tag like `@main` (instead of a version tag or commit SHA) means the Action's behavior can change without your knowledge — pin to a specific version, and prefer well-maintained, widely-used Actions.